

# SSEI-ROS2 Project Plan

## ROS 2-Based Space Station External Inspection Simulation and Safe Planning Framework

Version: 1.0

Target OS: Ubuntu 24.04.4 LTS

Target ROS: ROS 2 Jazzy Jalisco

Target Simulator: Gazebo Harmonic

Primary Use: Codex implementation, research simulation, paper experiment generation

---

## 1. Project Overview

### 1.1 Project Name

#### SSEI-ROS2: Space Station External Inspection with ROS 2

This project builds a complete ROS 2 simulation framework for autonomous spacecraft inspection of large orbital structures, using a simplified space station external structure as the initial target.

The framework is designed for:

1. Building a complete simulation environment.
  2. Visualizing spacecraft inspection around a space station.
  3. Developing safety-constrained inspection planning algorithms.
  4. Recording trajectory, control, safety, coverage, and planner data.
  5. Producing paper-ready figures, tables, videos, and reproducible experiment logs.
  6. Supporting future integration with Basilisk for high-fidelity astrodynamics.
  7. Supporting future publication in aerospace, robotics, and control venues.
- 

## 2. Core Goal

The goal is not merely to create a Gazebo demo.

The goal is to build a **research-grade ROS 2 digital-twin-style simulation framework** for:

Autonomous safety-constrained external inspection of large orbital structures.

The final system should support:

1. A free-flying inspection spacecraft.
2. A simplified external space station model.

3. Relative orbital dynamics in LVLH frame.
4. RViz2 research visualization.
5. Gazebo Harmonic 3D scene visualization.
6. Inspection target generation.
7. Visibility and coverage evaluation.
8. Baseline waypoint inspection.
9. Greedy next-best-view inspection planning.
10. Safety monitor and safety filter.
11. CBF/QP or projection-based safe control.
12. Automatic CSV, JSON, rosbag2, figure, and video generation.
13. Monte Carlo experiment execution.
14. Future MPC/CBF/HJI/ADP safe planning algorithms.

---

## 3. Target Research Direction

After the simulation framework is established, the main research algorithm should be:

Safety-Constrained Next-Best-View Inspection Planning for Space Station External Structures

Recommended algorithmic direction:

High-level planner:

coverage-aware next-best-view viewpoint selection

Mid-level planner:

dynamically feasible trajectory generation

Low-level safety layer:

CBF-QP, projection-based safety filter, MPC safety layer, or reachability-based shield

Evaluation:

ROS 2 + Gazebo + RViz + Monte Carlo + paper-grade data logging

---

## 4. Target Publication Positioning

The project can support several paper types.

### 4.1 Aerospace GNC Paper

Possible journals:

AIAA Journal of Guidance, Control, and Dynamics  
Aerospace Science and Technology

Acta Astronautica  
IEEE Transactions on Aerospace and Electronic Systems

Suitable contribution:

A safety-constrained spacecraft inspection trajectory planning algorithm with relative orbital dynamics, fuel constraints, visibility constraints, and collision avoidance.

## 4.2 Aerospace System / Digital Twin Paper

Possible journals:

AIAA Journal of Aerospace Information Systems  
Acta Astronautica  
IEEE Transactions on Aerospace and Electronic Systems  
SoftwareX  
JOSS

Suitable contribution:

A reproducible ROS 2 digital-twin framework for autonomous space station external inspection and safety verification.

## 4.3 Robotics Planning Paper

Possible journals/conferences:

IEEE Robotics and Automation Letters  
Robotics and Autonomous Systems  
Autonomous Robots  
IEEE Transactions on Robotics, only if the algorithm is very strong

Suitable contribution:

A general safety-aware next-best-view or coverage planning algorithm for large complex 3D structures.

## 5. System Assumptions

### 5.1 Operating System

Ubuntu 24.04.4 LTS

### 5.2 Core Software Stack

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| ROS 2:            | Jazzy Jalisco                                                               |
| Simulator:        | Gazebo Harmonic                                                             |
| Visualization:    | RViz2 + Gazebo GUI                                                          |
| Build tool:       | colcon                                                                      |
| ROS build system: | ament_cmake + ament_python                                                  |
| Language:         | Python 3.12 for high-level modules<br>C++ optional for later Gazebo plugins |
| Optimization:     | NumPy, SciPy, optional CVXPY/OSQP/CasADi                                    |
| Data processing:  | pandas, matplotlib, rosbag2                                                 |
| Dynamics:         | ROS-native HCW/CW first<br>Basilisk backend later                           |

### 5.3 Important Compatibility Decision

Do **not** base the project on:

ROS 1  
ROS 2 Humble  
Gazebo Classic  
catkin

This project is designed for:

ROS 2 Jazzy + Gazebo Harmonic

Existing ROS 2 Humble / Gazebo Classic ISS inspection repositories may be used as conceptual references only.

## 6. Main Deliverables

The project is complete only when the following deliverables exist.

## 6.1 Simulation Deliverables

1. ROS 2 workspace builds successfully.
2. Gazebo Harmonic opens with an ISS proxy model.
3. RViz2 opens with full research visualization.
4. A free-flying chaser spacecraft moves around the station.
5. Relative dynamics are simulated in LVLH frame.
6. Baseline controller tracks waypoints.
7. Inspection targets are generated and visualized.
8. Coverage increases as the spacecraft observes targets.
9. Safety monitor detects keep-out and collision constraints.
10. Safety filter modifies unsafe commands.

## 6.2 Paper Data Deliverables

1. CSV trajectory logs.
2. CSV control logs.
3. CSV safety logs.
4. CSV coverage logs.
5. CSV planner logs.
6. JSON summary file.
7. YAML configuration snapshot.
8. rosbag2 recording.
9. Publication-ready figures.
10. RViz/Gazebo screenshots or videos.
11. Monte Carlo summary table.
12. Method comparison figures.

## 6.3 One-Command Experiment

The final paper experiment should run with:

```
ros2 launch ssei_bringup paper_experiment.launch.py
  scenario:=full_station
  method:=safe_nbv_cbf
  record:=true
  save_figures:=true
  run_id:=exp001
```

Expected result:

```
data/results/<timestamp>/
├── config_snapshot/
├── raw/
├── rosbag/
├── figures/
```

```
|— videos/
|— summary.json
|— summary.md
```

## 7. Repository Structure

Create the workspace:

```
~/ssei_ws/
|— src/
|   |— ssei_bringup/
|   |— ssei_description/
|   |— ssei_gazebo/
|   |— ssei_interfaces/
|   |— ssei_dynamics/
|   |— ssei_control/
|   |— ssei_safety/
|   |— ssei_guidance/
|   |— ssei_perception/
|   |— ssei_mission/
|   |— ssei_eval/
|   |— ssei_utils/
|— scripts/
|   |— setup_ubuntu24_ros2_jazzy.sh
|   |— install_python_deps.sh
|   |— build_all.sh
|   |— run_demo.sh
|   |— record_paper_experiment.sh
|— docs/
|   |— architecture.md
|   |— dynamics_model.md
|   |— safety_constraints.md
|   |— visualization_requirements.md
|   |— logging_requirements.md
|   |— experiment_protocol.md
|   |— paper_experiment_design.md
|   |— codex_tasks.md
|— data/
|   |— scenarios/
|   |— logs/
|   |— bags/
|   |— results/
|— notebooks/
|   |— analyze_results.ipynb
|   |— plot_coverage.ipynb
|   |— compare_methods.ipynb
|— README.md
```

## 8. Package Responsibilities

### 8.1 ssei\_interfaces

Purpose:

Define custom ROS 2 messages, services, and actions.

Required messages:

InspectionTarget.msg  
InspectionTargetArray.msg  
InspectionWaypoint.msg  
InspectionWaypointArray.msg  
InspectionStatus.msg  
SafetyStatus.msg  
ControlCommand.msg  
CoverageMap.msg  
PlannerStatus.msg  
MissionSummary.msg

Required services:

ResetScenario.srv  
ComputeInspectionPath.srv  
EvaluateCoverage.srv  
SaveExperiment.srv

Required actions:

ExecuteInspection.action

Example:

```
# ControlCommand.msg
std_msgs/Header header
geometry_msgs/Vector3 acceleration_lvlh
geometry_msgs/Vector3 force_lvlh
geometry_msgs/Vector3 torque_body
```

```
float64 throttle_norm
bool is_safe_command
string source
```

Example:

```
# SafetyStatus.msg
std_msgs/Header header
bool is_safe
bool is_warning
bool safety_filter_active
float64 min_distance
float64 safety_margin
float64 cbf_value
string active_constraint
string message
```

Example:

```
# InspectionTarget.msg
std_msgs/Header header
string target_id
geometry_msgs/Point position_lvlh
geometry_msgs/Vector3 normal_lvlh
bool inspected
int32 inspection_count
float64 last_seen_time
float64 view_quality
```

---

## 8.2 ssei\_description

Purpose:

Store space station proxy model, chaser spacecraft model, meshes, URDF/SDF, and RViz configuration.

Structure:

```
ssei_description/
├── models/
│   ├── iss_proxy/
│   │   ├── model.sdf
│   │   ├── model.config
│   │   └── meshes/
```



```

├── chaser/
│   ├── model.sdf
│   ├── model.config
│   └── meshes/
├── inspection_target_marker/
├── urdf/
│   └── chaser.urdf.xacro
├── rviz/
│   ├── ssei_research.rviz
│   └── ssei_paper.rviz
└── package.xml

```

Initial ISS proxy model:

Main truss:

```
box center = [0, 0, 0]
size = [80, 4, 4] m
```

Left solar array:

```
box center = [-25, 0, 12]
size = [30, 1, 12] m
```

Right solar array:

```
box center = [25, 0, 12]
size = [30, 1, 12] m
```

## Module 1:

```
cylinder center = [0, 8, 0]
radius = 3 m
length = 15 m
axis = y
```

## Module 2:

```
cylinder center = [0, -8, 0]
radius = 3 m
length = 15 m
axis = y
```

Docking node:

```
cylinder center = [0, 0, -6]
radius = 2 m
length = 6 m
axis = z
```

Chaser initial model:

mass = 12 kg  
body shape = small cube or cylinder  
body size = approximately 0.6 m

camera mounted on front face  
optional thruster markers

## 8.3 ssei\_gazebo

Purpose:

Provide Gazebo Harmonic worlds, model spawning, ROS-Gazebo bridge, and scene visualization.

Structure:

```
ssei_gazebo/  
├── worlds/  
│   ├── iss_external_empty.sdf  
│   ├── iss_external_sunlight.sdf  
│   ├── iss_external_obstacles.sdf  
│   └── iss_external_paper.sdf  
├── launch/  
│   ├── gazebo_iss.launch.py  
│   ├── spawn_chaser.launch.py  
│   ├── bridge.launch.py  
│   └── gazebo_visualization.launch.py  
├── config/  
│   └── ros_gz_bridge.yaml  
└── package.xml
```

Important design rule:

Gazebo is used for 3D scene visualization and optional sensor rendering.  
The ROS dynamics node is the source of truth for spacecraft state in the early stages.

Do not rely on Gazebo physics as the initial spacecraft dynamics backend.

## 8.4 ssei\_dynamics

Purpose:

Provide relative orbital dynamics and state propagation.

Structure:

```

ssei_dynamics/
├── ssei_dynamics/
│   ├── __init__.py
│   ├── double_integrator.py
│   ├── hcw_dynamics.py
│   ├── nonlinear_relative_dynamics.py
│   ├── dynamics_node.py
│   ├── basilisk_adapter.py
│   └── frames.py
├── launch/
│   └── dynamics.launch.py
├── config/
│   └── dynamics.yaml
├── test/
│   ├── test_hcw_dynamics.py
│   └── test_frame_transforms.py
└── package.xml

```

State:

$$\mathbf{x} = [r_x, r_y, r_z, v_x, v_y, v_z]^T$$

where:

$r$  = relative position in LVLH frame  
 $v$  = relative velocity in LVLH frame

HCW dynamics:

$$\begin{aligned} \dot{r}_x &= v_x \\ \dot{r}_y &= v_y \\ \dot{r}_z &= v_z \\ \\ \dot{v}_x &= 3n^2 r_x + 2n v_y + a_x \\ \dot{v}_y &= -2n v_x + a_y \\ \dot{v}_z &= -n^2 r_z + a_z \end{aligned}$$

Parameters:

$$\begin{aligned} \mu &= 3.986004418e14 \text{ m}^3/\text{s}^2 \\ R_E &= 6378137.0 \text{ m} \\ h &= 400000.0 \text{ m} \\ a_{\text{ref}} &= R_E + h \\ n &= \sqrt[3]{\mu / a_{\text{ref}}} \end{aligned}$$

Default initial condition:

```
position_lvlh = [0.0, -35.0, 10.0] m  
velocity_lvlh = [0.0, 0.0, 0.0] m/s
```

Publish:

```
/chaser/odom  
/chaser/state_lvlh  
/tf
```

Subscribe:

```
/chaser/safe_control_command
```

---

## 8.5 ssei\_control

Purpose:

```
Provide baseline waypoint tracking controllers.
```

Structure:

```
ssei_control/  
├── ssei_control/  
│   ├── __init__.py  
│   ├── pid_controller.py  
│   ├── lqr_controller.py  
│   ├── mpc_controller.py  
│   ├── controller_node.py  
│   └── thruster_allocator.py  
├── launch/  
│   └── control.launch.py  
├── config/  
│   ├── pid.yaml  
│   ├── lqr.yaml  
│   └── mpc.yaml  
├── test/  
│   └── test_lqr_controller.py  
└── package.xml
```

Baseline controller:

```
u_nom = -K (x - x_ref)
```

Default constraints:

```
mass = 12.0 kg  
max_acceleration = 0.01 m/s^2  
max_speed = 0.2 m/s
```

Publish:

```
/chaser/control_command
```

Subscribe:

```
/chaser/odom  
/chaser/reference  
/inspection/current_waypoint
```

---

## 8.6 ssei\_safety

Purpose:

```
Provide safety monitor, keep-out zone checking, collision checking, and safety filter.
```

Structure:

```
ssei_safety/  
├── ssei_safety/  
│   ├── __init__.py  
│   ├── keepout_zones.py  
│   ├── primitive_geometry.py  
│   ├── collision_checker.py  
│   ├── cbf_filter.py  
│   ├── projection_filter.py  
│   ├── passive_safety.py  
│   ├── safety_monitor_node.py  
│   └── safety_filter_node.py  
├── config/  
│   └── safety.yaml  
├── test/  
│   ├── test_keepout_zones.py  
│   └── test_collision_checker.py
```

```
|   └── test_cbf_filter.py
|   └── package.xml
```

Safety constraints:

1. Minimum distance from station structure.
2. Warning distance from station structure.
3. Maximum relative speed.
4. Maximum acceleration.
5. Keep-out zone avoidance.
6. Field-of-view feasibility.
7. Passive safety horizon.
8. Collision-free waypoint transfer.
9. Control saturation.
10. Emergency hold state.

Keep-out primitive representation:

Box:

```
center: [x, y, z]
size: [lx, ly, lz]
safety_margin: d_safe
```

Cylinder:

```
center: [x, y, z]
radius: r
length: L
axis: [1, 0, 0] or [0, 1, 0] or [0, 0, 1]
safety_margin: d_safe
```

CBF concept:

```
h(x) = d(x, obstacle) - d_safe
safe set: h(x) >= 0
```

Safety filter problem:

```
minimize ||u - u_nom||^2
subject to h_dot(x, u) + alpha h(x) >= 0
          ||u|| <= u_max
```

For the first version, a projection-based filter is acceptable if CBF-QP takes longer to implement.

Publish:

```
/chaser/safety_status  
/chaser/safe_control_command  
/visualization/safety_markers
```

Subscribe:

```
/chaser/odom  
/chaser/control_command
```

---

## 8.7 ssei\_guidance

Purpose:

Provide inspection waypoint generation, next-best-view planning, and trajectory planning.

Structure:

```
ssei_guidance/  
├── ssei_guidance/  
│   ├── __init__.py  
│   ├── waypoint_generator.py  
│   ├── fixed_waypoint_planner.py  
│   ├── greedy_nbv_planner.py  
│   ├── coverage_planner.py  
│   ├── corridor_planner.py  
│   ├── inspection_planner_node.py  
│   └── path_smoother.py  
├── config/  
│   └── planner.yaml  
├── test/  
│   ├── test_waypoint_generator.py  
│   ├── test_candidate_sampling.py  
│   └── test_coverage_metric.py  
└── package.xml
```

Baseline planners:

1. Fixed waypoint planner.
2. Random safe waypoint planner.
3. Nearest-target greedy planner.
4. Greedy next-best-view planner.
5. Safe NBV planner with safety score.

Candidate score:

```
score = w_cov * new_coverage
        - w_dist * travel_distance
        - w_fuel * estimated_delta_v
        + w_safe * safety_margin
        + w_view * view_quality
```

Publish:

```
/inspection/waypoints
/inspection/current_waypoint
/inspection/planned_path
/planner/status
/visualization/planner_markers
```

Subscribe:

```
/chaser/odom
/inspection/coverage_map
/chaser/safety_status
```

---

## 8.8 ssei\_perception

Purpose:

Generate inspection targets, evaluate visibility, and maintain coverage map.

Structure:

```
ssei_perception/
├── ssei_perception/
│   ├── __init__.py
│   ├── camera_model.py
│   ├── visibility_checker.py
│   ├── coverage_map.py
│   ├── inspection_target_manager.py
│   ├── coverage_node.py
│   └── target_marker_node.py
├── config/
│   └── camera.yaml
├── test/
│   └── test_visibility_checker.py
```



```
|   └── test_coverage_map.py
|   └── package.xml
```

A target is inspected if:

1. Distance is within  $[r_{\min}, r_{\max}]$ .
2. Target is inside the camera field of view.
3. Viewing angle is below  $\theta_{\max}$ .
4. Line of sight is not blocked by station primitive geometry.
5. Target is observed longer than  $\text{dwell\_time}$ .

Default camera:

```
horizontal_fov = 70 deg
vertical_fov = 50 deg
r_min = 2.0 m
r_max = 25.0 m
theta_max = 60 deg
dwell_time = 1.0 s
```

Publish:

```
/inspection/targets
/inspection/coverage_map
/inspection/coverage_ratio
/visualization/target_markers
/visualization/fov_marker
```

Subscribe:

```
/chaser/odom
/tf
```

---

## 8.9 ssei\_mission

Purpose:

Coordinate scenario loading, mission state machine, execution, replanning, and experiment completion.

Structure:

```

ssei_mission/
├── ssei_mission/
│   ├── __init__.py
│   ├── mission_manager_node.py
│   ├── scenario_loader.py
│   ├── state_machine.py
│   ├── failure_injection.py
│   └── experiment_manager.py
├── config/
│   ├── mission_default.yaml
│   └── scenarios.yaml
├── test/
│   └── test_scenario_loader.py
└── package.xml

```

Mission states:

```

IDLE
INITIALIZE
PLAN
EXECUTE
HOLD
REPLAN
COMPLETE
ABORT

```

Publish:

```

/mission/status
/mission/event
/mission/summary

```

Subscribe:

```

/chaser/odom
/chaser/safety_status
/inspection/coverage_ratio
/planner/status

```

---

## 8.10 ssei\_eval

Purpose:

Record data, save rosbag, compute metrics, generate figures, and produce paper-ready summaries.

Structure:

```
ssei_eval/  
├── ssei_eval/  
│   ├── __init__.py  
│   ├── logger_node.py  
│   ├── metrics.py  
│   ├── rosbag_manager.py  
│   ├── plot_results.py  
│   ├── monte_carlo_runner.py  
│   ├── report_generator.py  
│   └── experiment_index.py  
├── config/  
│   └── eval.yaml  
├── test/  
│   ├── test_metrics.py  
│   └── test_logger_format.py  
└── package.xml
```

Required output:

```
data/results/<timestamp>/  
├── config_snapshot/  
│   ├── dynamics.yaml  
│   ├── planner.yaml  
│   ├── safety.yaml  
│   ├── camera.yaml  
│   ├── mission.yaml  
│   └── scenario.yaml  
├── raw/  
│   ├── trajectory.csv  
│   ├── control.csv  
│   ├── coverage.csv  
│   ├── safety.csv  
│   ├── planner.csv  
│   └── mission_events.csv  
├── rosbag/  
│   └── ssei_run/  
├── figures/  
│   ├── trajectory_3d.png  
│   ├── position_over_time.png  
│   ├── velocity_over_time.png  
│   ├── coverage_over_time.png  
│   ├── min_distance_over_time.png  
│   └── control_effort_over_time.png
```

```

|   |—— cumulative_delta_v_over_time.png
|   |—— safety_margin_over_time.png
|   |—— method_comparison.png
|—— videos/
|   |—— rviz_recording.mp4
|   |—— gazebo_recording.mp4
|—— summary.json
|—— summary.md

```

## 8.11 ssei\_bringup

Purpose:

Provide top-level launch files for demos, debugging, and paper experiments.

Structure:

```

ssei_bringup/
|—— launch/
|   |—— demo_rviz.launch.py
|   |—— demo_gazebo.launch.py
|   |—— demo_fixed_waypoints.launch.py
|   |—— demo_coverage.launch.py
|   |—— demo_greedy_inspection.launch.py
|   |—— demo_safety_filter.launch.py
|   |—— paper_experiment.launch.py
|   |—— monte_carlo_experiment.launch.py
|—— config/
|   |—— bringup_default.yaml
|—— package.xml

```

# 9. ROS Graph

## 9.1 Main Topics

|                              |                                       |
|------------------------------|---------------------------------------|
| /chaser/odom                 | nav_msgs/Odometry                     |
| /chaser/state_lv1h           | nav_msgs/Odometry                     |
| /chaser/reference            | nav_msgs/Odometry                     |
| /chaser/control_command      | ssei_interfaces/ControlCommand        |
| /chaser/safe_control_command | ssei_interfaces/ControlCommand        |
| /chaser/safety_status        | ssei_interfaces/SafetyStatus          |
| /inspection/targets          | ssei_interfaces/InspectionTargetArray |

|                                  |                                         |
|----------------------------------|-----------------------------------------|
| /inspection/coverage_map         | ssei_interfaces/CoverageMap             |
| /inspection/coverage_ratio       | std_msgs/Float64                        |
| /inspection/waypoints            | ssei_interfaces/InspectionWaypointArray |
| /inspection/planned_path         | nav_msgs/Path                           |
| /inspection/current_waypoint     | ssei_interfaces/InspectionWaypoint      |
|                                  |                                         |
| /planner/status                  | ssei_interfaces/PlannerStatus           |
| /mission/status                  | std_msgs/String                         |
| /mission/event                   | std_msgs/String                         |
| /mission/summary                 | ssei_interfaces/MissionSummary          |
|                                  |                                         |
| /visualization/target_markers    | visualization_msgs/MarkerArray          |
| /visualization/safety_markers    | visualization_msgs/MarkerArray          |
| /visualization/planner_markers   | visualization_msgs/MarkerArray          |
| /visualization/fov_marker        | visualization_msgs/Marker               |
| /visualization/trajectory_marker | visualization_msgs/Marker               |
|                                  |                                         |
| /tf                              | tf2_msgs/TFMessage                      |
| /tf_static                       | tf2_msgs/TFMessage                      |

## 9.2 Main Node List

```

dynamics_node
controller_node
safety_monitor_node
safety_filter_node
inspection_planner_node
coverage_node
target_marker_node
mission_manager_node
logger_node
rosvbag_manager_node
rviz_marker_node
gazebo_bridge_node

```

## 9.3 Data Flow

```

mission_manager
-> inspection_planner
    -> waypoint list / planned path
    -> controller
        -> nominal control command
        -> safety_filter
            -> safe control command
            -> dynamics_node
                -> odometry
                -> coverage_node
                -> safety_monitor

```

-> logger\_node  
-> RViz visualization  
-> Gazebo visualization

## 10. TF Frame Convention

Use the following TF tree:

```
world
├── lvlh
│   ├── iss_body
│   └── chaser_body
│       └── chaser_camera
```

Frame definitions:

**world:**  
visualization frame; initially aligned with LVLH.

**lvlh:**  
local vertical local horizontal frame centered at the reference station orbit.

**iss\_body:**  
body-fixed frame of the station proxy.

**chaser\_body:**  
body frame of the inspection spacecraft.

**chaser\_camera:**  
optical frame of the inspection camera.

Initial implementation:

world and lvlh may be identical.

Future extension:

Add ECI, ECEF, orbital frame, and station attitude frame.

# 11. Visualization Requirements

Visualization is a hard deliverable, not optional.

## 11.1 RViz2 Research Visualization

RViz2 must show:

1. ISS proxy model.
2. Chaser spacecraft model.
3. TF frames.
4. Chaser current pose.
5. Executed trajectory.
6. Planned path.
7. Current waypoint.
8. Inspection targets.
9. Uninspected targets in one color.
10. Inspected targets in another color.
11. Keep-out zones.
12. Warning zones.
13. Camera FOV cone.
14. Minimum-distance text marker.
15. Safety status marker.
16. Coverage ratio text marker.
17. Control command vector.
18. Optional velocity vector.

RViz config files:

```
ssei_description/rviz/ssei_research.rviz  
ssei_description/rviz/ssei_paper.rviz
```

RViz launch command:

```
ros2 launch ssei_bringup demo_rviz.launch.py
```

Expected result:

RViz opens and shows the chaser, station proxy, TF tree, planned path, executed trajectory, targets, coverage state, and safety markers.

## 11.2 Gazebo Harmonic Visualization

Gazebo must show:

1. ISS proxy external structure.
2. Free-flying chaser spacecraft.
3. Camera/depth sensor model if available.
4. Lighting and simple space background.
5. Optional obstacles.
6. Smooth visual update of chaser pose.

Gazebo launch command:

```
ros2 launch ssei_bringup demo_gazebo.launch.py
```

Important:

Gazebo visualization should follow the ROS dynamics state.  
ROS dynamics remains the source of truth during early phases.

## 11.3 Video and Screenshot Requirements

The project should support:

1. RViz screenshot.
2. Gazebo screenshot.
3. RViz video recording.
4. Gazebo video recording.
5. Replaying experiment from rosbag2.

Video output:

```
data/results/<timestamp>/videos/  
├── rviz_recording.mp4  
└── gazebo_recording.mp4
```

---

## 12. Logging and Paper Data Requirements

Data logging is a hard deliverable, not optional.

### 12.1 Trajectory Data

Record every time step:



```
time
rx, ry, rz
vx, vy, vz
qx, qy, qz, qw
wx, wy, wz
target_rx, target_ry, target_rz
tracking_error_norm
```

Save to:

```
raw/trajectory.csv
```

Paper figures:

```
3D trajectory
position over time
velocity over time
tracking error over time
method trajectory comparison
```

## 12.2 Control Data

Record:

```
time
ax_nom, ay_nom, az_nom
ax_safe, ay_safe, az_safe
force_x, force_y, force_z
torque_x, torque_y, torque_z
control_norm
acceleration_norm
delta_v_increment
cumulative_delta_v
is_saturated
```

Save to:

```
raw/control.csv
```

Paper figures:

```
control input over time
control effort comparison
```

cumulative delta-v comparison  
saturation count

## 12.3 Coverage Data

Record:

time  
total\_targets  
inspected\_targets  
coverage\_ratio  
new\_targets\_seen  
mean\_viewing\_angle  
mean\_inspection\_range  
visible\_target\_count

Save to:

raw/coverage.csv

Paper figures:

coverage over time  
final coverage comparison  
coverage speed comparison  
target-level inspection map

## 12.4 Safety Data

Record:

time  
min\_distance\_to\_station  
warning\_distance  
safety\_margin  
cbf\_value  
active\_constraint  
safety\_filter\_active  
num\_safety\_violations  
relative\_speed  
passive\_safety\_status

Save to:

```
raw/safety.csv
```

Paper figures:

```
minimum distance over time  
safety margin over time  
CBF value over time  
safety violation comparison
```

## 12.5 Planner Data

Record:

```
time  
planner_type  
current_waypoint_id  
num_candidate_viewpoints  
selected_viewpoint_score  
coverage_gain  
distance_cost  
fuel_cost  
safety_score  
view_quality_score  
planner_runtime_ms  
replan_triggered
```

Save to:

```
raw/planner.csv
```

Paper figures:

```
planner runtime comparison  
score component analysis  
ablation study  
replanning statistics
```

## 12.6 Mission Event Data

Record:

```
time  
event_type
```

```
state
message
coverage_ratio
min_distance
current_waypoint_id
```

Save to:

```
raw/mission_events.csv
```

## 12.7 Summary JSON

At the end of each run, save:

```
{
  "scenario_name": "full_station",
  "method_name": "safe_nbv_cbf",
  "random_seed": 1,
  "success": true,
  "final_coverage_ratio": 0.85,
  "mission_time": 650.0,
  "total_delta_v": 1.25,
  "minimum_distance": 2.35,
  "number_of_safety_violations": 0,
  "number_of_replans": 3,
  "mean_planner_runtime_ms": 18.2,
  "mean_controller_runtime_ms": 1.1,
  "git_commit_hash": "unknown"
}
```

Save to:

```
summary.json
```

## 12.8 Summary Markdown

Generate:

```
summary.md
```

Content:

1. Scenario name.
2. Method name.

3. Configuration snapshot path.
4. Key metrics.
5. Generated figures.
6. rosbag path.
7. Notes on safety violations.
8. Notes on mission success or failure.

---

## 13. rosbag2 Recording

The paper experiment must record:

```
ros2 bag record
/chaser/odom
/chaser/reference
/chaser/control_command
/chaser/safe_control_command
/chaser/safety_status
/inspection/targets
/inspection/coverage_map
/inspection/coverage_ratio
/inspection/waypoints
/inspection/planned_path
/planner/status
/mission/status
/mission/event
/tf
/tf_static
```

Save to:

```
data/results/<timestamp>/rosbag/ssei_run/
```

Purpose:

1. Replay experiments.
2. Generate videos after the run.
3. Debug planner/control/safety interactions.
4. Preserve full ROS state for paper reproducibility.

---

## 14. Experiment Output Structure

Every run must produce:

```
data/results/<timestamp>/
├── config_snapshot/
│   ├── dynamics.yaml
│   ├── planner.yaml
│   ├── safety.yaml
│   ├── camera.yaml
│   ├── mission.yaml
│   └── scenario.yaml
├── raw/
│   ├── trajectory.csv
│   ├── control.csv
│   ├── coverage.csv
│   ├── safety.csv
│   ├── planner.csv
│   └── mission_events.csv
├── rosbag/
│   └── ssei_run/
├── figures/
│   ├── trajectory_3d.png
│   ├── position_over_time.png
│   ├── velocity_over_time.png
│   ├── coverage_over_time.png
│   ├── min_distance_over_time.png
│   ├── control_effort_over_time.png
│   ├── cumulative_delta_v_over_time.png
│   ├── safety_margin_over_time.png
│   └── method_comparison.png
├── videos/
│   ├── rviz_recording.mp4
│   └── gazebo_recording.mp4
├── summary.json
└── summary.md
```

Do not consider a paper experiment successful unless this directory is generated.

---

## 15. Installation Script

Create:

```
scripts/setup_ubuntu24_ros2_jazzy.sh
```

Content:

```
#!/usr/bin/env bash
set -e
```

```

echo "[SSEI] Updating apt..."
sudo apt update
sudo apt install -y software-properties-common curl gnupg lsb-release locales

echo "[SSEI] Setting locale..."
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

echo "[SSEI] Enabling universe repository..."
sudo add-apt-repository -y universe

echo "[SSEI] Installing ROS 2 apt key..."
sudo mkdir -p /usr/share/keyrings
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key
-o /usr/share/keyrings/ros-archive-keyring.gpg

echo "[SSEI] Adding ROS 2 repository..."
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $
{UBUNTU_CODENAME}) main"
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

echo "[SSEI] Installing ROS 2 Jazzy and Gazebo integration..."
sudo apt update
sudo apt install -y
ros-jazzy-desktop
ros-jazzy-ros-gz
ros-dev-tools
python3-colcon-common-extensions
python3-rosdep
python3-vcstool
python3-pip
python3-venv
build-essential
cmake
git
wget
unzip
pkg-config

echo "[SSEI] Installing useful ROS 2 packages..."
sudo apt install -y
ros-jazzy-tf2-tools
ros-jazzy-rviz2
ros-jazzy-robot-state-publisher
ros-jazzy-joint-state-publisher
ros-jazzy-xacro
ros-jazzy-ros-gz-bridge
ros-jazzy-ros-gz-sim
ros-jazzy-ros-gz-image

```

```

ros-jazzy-image-transport
ros-jazzy-cv-bridge
ros-jazzy-vision-msgs
ros-jazzy-visualization-msgs
ros-jazzy-diagnostic-msgs
ros-jazzy-rosbag2
ros-jazzy-rosbag2-storage-mcap
ros-jazzy-rosbag2-transport

echo "[SSEI] Installing Python scientific packages..."
sudo apt install -y
python3-numpy
python3-scipy
python3-matplotlib
python3-pandas
python3-yaml
python3-pytest
python3-opencv
python3-sympy

echo "[SSEI] Initializing rosdep..."
if [ ! -f /etc/ros/rosdep/sources.list.d/20-default.list ]; then
    sudo rosdep init || true
fi
rosdep update

echo "[SSEI] Adding ROS setup to bashrc..."
if ! grep -q "source /opt/ros/jazzy/setup.bash" ~/.bashrc; then
    echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
fi

echo "[SSEI] Done."
echo "Run: source /opt/ros/jazzy/setup.bash"

```

Run:

```

chmod +x scripts/setup_ubuntu24_ros2_jazzy.sh
./scripts/setup_ubuntu24_ros2_jazzy.sh
source /opt/ros/jazzy/setup.bash

```

Verify:

```

ros2 doctor
ros2 run demo_nodes_cpp talker

```

In another terminal:



```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

Verify Gazebo:

```
ros2 launch ros_gz_sim gz_sim.launch.py gz_args:="shapes.sdf"
```

---

## 16. Workspace Creation

Run:

```
mkdir -p ~/ssei_ws/src
cd ~/ssei_ws
mkdir -p scripts docs data/scenarios data/logs data/bags data/results notebooks
source /opt/ros/jazzy/setup.bash
```

Create packages:

```
cd ~/ssei_ws/src

ros2 pkg create ssei_interfaces --build-type ament_cmake
--dependencies std_msgs geometry_msgs nav_msgs action_msgs builtin_interfaces
rosidl_default_generators visualization_msgs

ros2 pkg create ssei_description --build-type ament_cmake

ros2 pkg create ssei_gazebo --build-type ament_python
--dependencies rclpy geometry_msgs nav_msgs sensor_msgs tf2_ros ros_gz_bridge ros_gz_sim
visualization_msgs

ros2 pkg create ssei_dynamics --build-type ament_python
--dependencies rclpy geometry_msgs nav_msgs std_msgs tf2_ros ssei_interfaces

ros2 pkg create ssei_control --build-type ament_python
--dependencies rclpy geometry_msgs nav_msgs std_msgs ssei_interfaces

ros2 pkg create ssei_safety --build-type ament_python
--dependencies rclpy geometry_msgs nav_msgs std_msgs visualization_msgs ssei_interfaces

ros2 pkg create ssei_guidance --build-type ament_python
--dependencies rclpy geometry_msgs nav_msgs std_msgs visualization_msgs ssei_interfaces
ssei_safety

ros2 pkg create ssei_perception --build-type ament_python
```

```
--dependencies rclpy geometry_msgs nav_msgs sensor_msgs std_msgs visualization_msgs  
ssei_interfaces
```

```
ros2 pkg create ssei_mission --build-type ament_python  
--dependencies rclpy geometry_msgs nav_msgs std_msgs ssei_interfaces
```

```
ros2 pkg create ssei_eval --build-type ament_python  
--dependencies rclpy geometry_msgs nav_msgs std_msgs ssei_interfaces
```

```
ros2 pkg create ssei_bringup --build-type ament_python  
--dependencies rclpy launch launch_ros
```

```
ros2 pkg create ssei_utils --build-type ament_python  
--dependencies rclpy geometry_msgs nav_msgs std_msgs
```

Build:

```
cd ~/ssei_ws  
rosdep install --from-paths src -y --ignore-src  
colcon build --symlink-install  
source install/setup.bash
```

Add workspace setup:

```
echo "source ~/ssei_ws/install/setup.bash" >> ~/.bashrc
```

---

## 17. Configuration Files

### 17.1 `dynamics.yaml`

```
dynamics:  
  model: "hbw"  
  mu: 3.986004418e14  
  earth_radius: 6378137.0  
  orbit_altitude: 400000.0  
  mass: 12.0  
  max_acceleration: 0.01  
  integration_dt: 0.02  
  publish_rate: 50.0  
  initial_state:  
    position: [0.0, -35.0, 10.0]  
    velocity: [0.0, 0.0, 0.0]
```

## 17.2 control.yaml

```
control:
  type: "lqr"
  publish_rate: 50.0
  max_acceleration: 0.01
  max_speed: 0.2
  waypoint_tolerance: 0.5
  velocity_tolerance: 0.02
  lqr:
    q_position: 10.0
    q_velocity: 2.0
    r_control: 1.0
```

## 17.3 safety.yaml

```
safety:
  min_distance: 2.0
  warning_distance: 4.0
  max_speed: 0.2
  max_acceleration: 0.01
  passive_safety_horizon: 30.0
  passive_safety_dt: 1.0
  filter_type: "projection"
  cbf:
    enabled: false
    alpha: 0.5
    solver: "osqp"
```

## 17.4 planner.yaml

```
planner:
  type: "greedy_nbv"
  coverage_threshold: 0.8
  max_waypoints: 80
  candidate_radius_min: 8.0
  candidate_radius_max: 30.0
  candidate_count: 500
  weights:
    coverage: 10.0
    distance: 0.1
    fuel: 0.5
    safety: 2.0
    view_quality: 1.0
```

## 17.5 camera.yaml

```
camera:
  frame_id: "chaser_camera"
  horizontal_fov_deg: 70.0
  vertical_fov_deg: 50.0
  min_range: 2.0
  max_range: 25.0
  max_view_angle_deg: 60.0
  dwell_time: 1.0
```

## 17.6 eval.yaml

```
eval:
  result_root: "data/results"
  save_csv: true
  save_json: true
  save_figures: true
  record_rosbag: true
  save_config_snapshot: true
  save_git_commit_hash: true
  plot:
    dpi: 300
    figure_format: "png"
```

---

# 18. Build Phases

## Phase 1: Environment Bootstrap

Goal:

```
Install ROS 2 Jazzy, Gazebo Harmonic integration, and create the package skeleton.
```

Deliverables:

1. setup script.
2. workspace structure.
3. empty ROS 2 packages.
4. colcon build success.

Acceptance test:

```
cd ~/ssei_ws  
colcon build --symlink-install  
source install/setup.bash  
ros2 pkg list | grep ssei
```

---

## Phase 2: ROS-Only Dynamics + RViz Demo

Goal:

Run relative spacecraft motion without Gazebo physics.

Implement:

```
ssei_dynamics/hcw_dynamics.py  
ssei_dynamics/dynamics_node.py  
ssei_control/lqr_controller.py  
ssei_control/controller_node.py  
ssei_bringup/demo_rviz.launch.py
```

Acceptance test:

```
ros2 launch ssei_bringup demo_rviz.launch.py
```

Expected:

1. /chaser/odom is published.
2. TF contains lvlh -> chaser\_body.
3. RViz shows chaser trajectory.
4. Controller tracks a fixed waypoint.

---

## Phase 3: CSV Logger + Basic Paper Data

Goal:

Before adding complex planning, ensure trajectory and control can be recorded.

Implement:

```
ssei_eval/logger_node.py
ssei_eval/metrics.py
ssei_eval/plot_results.py
```

Acceptance test:

```
ros2 launch ssei_bringup demo_rviz.launch.py record:=true
```

Expected output:

```
data/results/<timestamp>/
├── raw/
│   ├── trajectory.csv
│   └── control.csv
├── figures/
│   ├── trajectory_3d.png
│   └── control_effort_over_time.png
└── summary.json
```

---

## Phase 4: ISS Proxy + Gazebo Visualization

Goal:

Visualize the station and chaser in Gazebo Harmonic.

Implement:

```
ssei_description/models/iss_proxy/model.sdf
ssei_description/models/chaser/model.sdf
ssei_gazebo/worlds/iss_external_empty.sdf
ssei_gazebo/launch/gazebo_iss.launch.py
ssei_bringup/demo_gazebo.launch.py
```

Acceptance test:

```
ros2 launch ssei_bringup demo_gazebo.launch.py
```

Expected:

1. Gazebo opens.
2. ISS proxy is visible.

3. Chaser is visible.
4. Chaser pose follows ROS state.
5. RViz and Gazebo show consistent state.

---

## Phase 5: Inspection Targets + Coverage Visualization

Goal:

Generate targets and show inspected/uninspected state.

Implement:

```
ssei_perception/inspection_target_manager.py  
ssei_perception/visibility_checker.py  
ssei_perception/coverage_map.py  
ssei_perception/coverage_node.py  
ssei_perception/target_marker_node.py
```

Acceptance test:

```
ros2 launch ssei_bringup demo_coverage.launch.py
```

Expected:

1. Targets appear in RViz.
2. Uninspected targets and inspected targets use different marker colors.
3. Coverage ratio is published.
4. Coverage.csv is saved.

---

## Phase 6: Fixed Waypoint Inspection Demo

Goal:

Complete one full inspection loop with fixed waypoints.

Implement:

```
ssei_guidance/fixed_waypoint_planner.py  
ssei_mission/mission_manager_node.py  
ssei_bringup/demo_fixed_waypoints.launch.py
```

Acceptance test:

```
ros2 launch ssei_bringup demo_fixed_waypoints.launch.py
```

Expected:

1. Chaser follows fixed inspection waypoints.
2. Coverage increases.
3. Safety monitor runs.
4. Data are saved automatically.
5. Figures are generated automatically.

---

## Phase 7: Safety Monitor + Safety Filter

Goal:

Add safety checking and command modification.

Implement:

```
ssei_safety/keepout_zones.py  
ssei_safety/collision_checker.py  
ssei_safety/safety_monitor_node.py  
ssei_safety/projection_filter.py  
ssei_safety/cbf_filter.py  
ssei_safety/safety_filter_node.py
```

Acceptance test:

```
ros2 launch ssei_bringup demo_safety_filter.launch.py
```

Expected:

1. Unsafe command is modified.
2. Minimum distance remains above safety margin.
3. Safety status is published.



4. Safety.csv is saved.
5. Safety figures are generated.

---

## Phase 8: Greedy Next-Best-View Planner

Goal:

Add autonomous coverage-aware viewpoint selection.

Implement:

```
ssei_guidance/waypoint_generator.py  
ssei_guidance/greedy_nbv_planner.py  
ssei_guidance/inspection_planner_node.py
```

Acceptance test:

```
ros2 launch ssei_bringup demo_greedy_inspection.launch.py
```

Expected:

1. Planner generates waypoints automatically.
2. Chaser follows waypoints.
3. Coverage reaches at least 80%.
4. Planner.csv is saved.
5. Coverage and trajectory figures are generated.

---

## Phase 9: Paper Experiment Mode

Goal:

Run one complete paper-style experiment with visualization, rosbag, CSV, figures, and summary.

Implement:

```
ssei_bringup/paper_experiment.launch.py  
ssei_eval/rosbag_manager.py
```

```
ssei_eval/report_generator.py  
ssei_eval/experiment_index.py
```

Acceptance test:

```
ros2 launch ssei_bringup paper_experiment.launch.py  
  scenario:=full_station  
  method:=greedy_nbv_safety_filter  
  record:=true  
  save_figures:=true  
  run_id:=exp001
```

Expected:

```
data/results/<timestamp>/  
├── config_snapshot/  
├── raw/  
├── rosbag/  
├── figures/  
├── videos/  
├── summary.json  
└── summary.md
```

---

## Phase 10: Monte Carlo and Method Comparison

Goal:

```
Support paper-level statistical comparison.
```

Implement:

```
ssei_eval/monte_carlo_runner.py  
ssei_eval/compare_methods.py
```

Acceptance test:

```
ros2 run ssei_eval monte_carlo_runner  
  --scenario full_station  
  --methods fixed_waypoints random_safe greedy_nbv greedy_nbv_safety_filter  
  --num-runs 20
```

Expected:

1. Multiple runs are completed.
2. Each run has its own output folder.
3. Combined summary table is generated.
4. Method comparison figure is generated.
5. Success rate is computed.

---

## Phase 11: Basilisk Backend

Goal:

Add optional high-fidelity spacecraft dynamics backend.

Implement:

```
ssei_dynamics/basilisk_adapter.py  
ssei_bringup/demo_basilisk_backend.launch.py
```

Rules:

1. Do not break existing ROS interfaces.
2. Keep `/chaser/odom` unchanged.
3. Keep `/chaser/safe_control_command` unchanged.
4. If Basilisk is not installed, fail gracefully.
5. Use launch argument `dynamics_backend:=hcw` or `basilisk`.

Acceptance test:

```
ros2 launch ssei_bringup demo_basilisk_backend.launch.py dynamics_backend:=basilisk
```

Expected:

1. Basilisk publishes chaser state.
2. Existing planner and controller still work.
3. HCW and Basilisk trajectories can be compared.

---

## 19. Minimum Viable Demo

The first complete demo is:

```
ros2 launch ssei_bringup demo_fixed_waypoints.launch.py
```

Required behavior:

1. RViz opens.
2. Gazebo opens.
3. ISS proxy is visible.
4. Chaser is visible.
5. Chaser tracks a sequence of inspection waypoints.
6. Safety monitor publishes safe status.
7. Coverage ratio increases.
8. Results are saved.

Minimum performance target:

```
coverage_ratio >= 0.80  
minimum_distance >= 2.0 m  
safety_violations = 0  
mission_time <= 1200 s simulated time
```

---

## 20. Paper Experiment Design

### 20.1 Scenarios

Use at least five scenarios:

```
Scenario A: simple truss inspection  
Scenario B: module inspection  
Scenario C: solar array inspection  
Scenario D: full station proxy inspection  
Scenario E: full station proxy with uncertainty
```

### 20.2 Methods

Compare:

1. Fixed waypoint inspection.
2. Random safe waypoint inspection.
3. Greedy nearest-target inspection.
4. Greedy next-best-view inspection.

5. Greedy next-best-view + safety filter.
6. Proposed safe NBV + CBF/MPC planner.

## 20.3 Metrics

Report:

```
coverage_ratio
mission_time
total_delta_v
minimum_distance
number_of_safety_violations
mean_view_quality
number_of_replans
planner_runtime
controller_runtime
success_rate
```

## 20.4 Monte Carlo

Run:

```
N = 20 initially
N = 100 for paper-quality final experiment
```

Randomize:

```
initial position
initial velocity
target distribution
sensor noise
actuation noise
geometry perturbation
```

---

## 21. Publication-Ready Figures

Automatically generate:

1. 3D trajectory around station.
2. Coverage over time.
3. Minimum distance over time.
4. Safety margin over time.
5. Control effort over time.

6. Cumulative delta-v over time.
7. Planner runtime comparison.
8. Method comparison bar chart.
9. Success rate table.
10. Representative RViz/Gazebo screenshots.

Figure style:

1. Use high DPI.
2. Use readable labels.
3. Use SI units.
4. Save both PNG and optional PDF.
5. Use consistent method names.
6. Include legends.
7. Avoid overly decorative colors.

---

## 22. Codex Development Rules

Codex must follow these rules:

1. Implement one package or one feature at a time.
  2. Never rewrite the whole repository unless explicitly requested.
  3. Keep ROS interfaces stable.
  4. Use YAML for parameters.
  5. Use Python type hints where reasonable.
  6. Add tests for mathematical modules.
  7. Add launch files for every demo.
  8. Keep dynamics independent from Gazebo physics in early phases.
  9. Keep RViz visualization working before adding complex algorithms.
  10. Add CSV logging before Monte Carlo.
  11. Add rosbag only after main topics are stable.
  12. Use colcon build --symlink-install after major changes.
  13. Use rosdep for system dependencies.
  14. Avoid hard-coded absolute paths.
  15. Save all experiment data automatically.
  16. Do not introduce ROS 1, Gazebo Classic, or ROS 2 Humble dependencies.
  17. Do not implement Basilisk before ROS-native HCW simulation works.
  18. Do not consider a milestone complete unless acceptance tests pass.
-

## 23. Codex Implementation Prompts

### Prompt 1: Repository Bootstrap

You are working in ~/ssei\_ws. Read PROJECT\_PLAN.md completely.

Implement Phase 1 only.

Create the ROS 2 Jazzy workspace package structure exactly as specified:  
ssei\_interfaces, ssei\_description, ssei\_gazebo, ssei\_dynamics, ssei\_control, ssei\_safety,  
ssei\_guidance, ssei\_perception, ssei\_mission, ssei\_eval, ssei\_bringup, ssei\_utils.

Add package.xml, setup.py or CMakeLists.txt as appropriate.

Add README.md files for each package.

Make sure colcon build --symlink-install succeeds.

Do not implement advanced algorithms yet.

Do not introduce ROS 1, Gazebo Classic, or ROS 2 Humble dependencies.

### Prompt 2: HCW Dynamics Node

Implement the HCW relative dynamics module in ssei\_dynamics.

Requirements:

1. Add hcw\_dynamics.py with class HCWDynamics.
2. State is [rx, ry, rz, vx, vy, vz].
3. Control input is acceleration [ax, ay, az].
4. Implement derivative and RK4 step.
5. Add tests for zero input and bounded propagation.
6. Add dynamics\_node.py that publishes nav\_msgs/Odometry on /chaser/odom.
7. Subscribe to /chaser/safe\_control\_command.
8. Load parameters from YAML.
9. Publish TF lvlh -> chaser\_body.
10. Add launch file dynamics.launch.py.
11. Ensure colcon build and pytest pass.

### Prompt 3: LQR Controller

Implement the baseline LQR waypoint tracking controller in ssei\_control.

Requirements:

1. Subscribe to /chaser/odom.
2. Subscribe to /chaser/reference.
3. Publish ssei\_interfaces/ControlCommand on /chaser/control\_command.
4. Implement continuous or discrete LQR for HCW dynamics.
5. Add acceleration saturation.

6. Add YAML config.
7. Add tests for command shape, saturation, and convergence direction.
8. Add launch file `control.launch.py`.

## Prompt 4: RViz Demo

Create a ROS-only RViz demo.

Requirements:

1. Launch `dynamics_node`.
2. Launch `controller_node`.
3. Publish a fixed reference waypoint.
4. Publish TF from `lvlh` to `chaser_body`.
5. Add RViz config showing trajectory, odometry, frames, and current waypoint.
6. Add `demo_rviz.launch.py` in `ssei_bringup`.
7. The command `ros2 launch ssei_bringup demo_rviz.launch.py` should show the chaser moving toward the waypoint.

## Prompt 5: CSV Logger

Implement paper-grade CSV logging in `ssei_eval`.

Requirements:

1. Add `logger_node.py`.
2. Record `trajectory.csv` from `/chaser/odom`.
3. Record `control.csv` from `/chaser/control_command` and `/chaser/safe_control_command`.
4. Create `data/results/<timestamp>/raw/`.
5. Create `summary.json`.
6. Add `plot_results.py` to generate `trajectory_3d.png` and `control_effort_over_time.png`.
7. Add launch argument `record:=true` to `demo_rviz.launch.py`.
8. Make sure data are saved automatically when the run ends.

## Prompt 6: ISS Proxy Model

Create a simplified ISS proxy model for Gazebo Harmonic and RViz.

Requirements:

1. Main truss as box.
2. Solar arrays as thin panels.
3. Pressurized modules as cylinders.
4. Add `model.sdf` and `model.config`.
5. Add Gazebo world `iss_external_empty.sdf`.
6. Add launch file `gazebo_iss.launch.py`.
7. Add `demo_gazebo.launch.py` in `ssei_bringup`.



8. The model should load in Gazebo using ROS 2 Jazzy and `ros_gz_sim`.
9. Chaser visual pose should follow `/chaser/odom`.

## Prompt 7: Inspection Targets and Coverage

Implement inspection target generation and coverage evaluation.

Requirements:

1. Generate target points on ISS proxy surfaces.
2. Publish targets as `visualization_msgs/MarkerArray`.
3. Implement visibility checking using range, FOV, viewing angle, and simplified occlusion.
4. Maintain inspected/uninspected status.
5. Publish coverage ratio.
6. Save `coverage.csv`.
7. Add `demo_coverage.launch.py`.
8. RViz should show uninspected targets and inspected targets with different colors.

## Prompt 8: Fixed Waypoint Mission

Implement a fixed waypoint inspection mission.

Requirements:

1. Add `fixed_waypoint_planner.py`.
2. Add `mission_manager_node.py`.
3. Generate a sequence of waypoints around the ISS proxy.
4. Controller should track each waypoint.
5. Mission should switch to next waypoint when tolerance is reached.
6. Stop when coverage threshold is reached or waypoint list ends.
7. Save trajectory, control, coverage, safety, and `mission_events` CSV files.
8. Generate figures automatically.
9. Add `demo_fixed_waypoints.launch.py`.

## Prompt 9: Safety Monitor and Filter

Implement a basic safety monitor and safety filter.

Requirements:

1. Compute minimum distance from chaser to ISS proxy primitives.
2. Publish `SafetyStatus`.
3. Implement projection-based safety filter first.
4. Add CBF-QP placeholder or optional implementation.
5. Subscribe to `/chaser/control_command`.
6. Publish `/chaser/safe_control_command`.
7. Enforce acceleration and speed limits.
8. Save `safety.csv`.

9. Add `demo_safety_filter.launch.py`.
10. Demonstrate that unsafe commands are modified.

## Prompt 10: Greedy NBV Planner

Implement a greedy next-best-view inspection planner.

Requirements:

1. Sample candidate viewpoints around ISS proxy.
2. Reject candidates inside keep-out zones.
3. Score candidates using coverage gain, distance cost, fuel estimate, safety margin, and view quality.
4. Generate waypoint sequence until coverage threshold is reached.
5. Publish waypoints as `nav_msgs/Path` and custom waypoint messages.
6. Save `planner.csv`.
7. Add tests for candidate generation and scoring.
8. Add `demo_greedy_inspection.launch.py`.

## Prompt 11: Paper Experiment Launch

Implement `paper_experiment.launch.py`.

Requirements:

1. Launch Gazebo.
2. Launch RViz.
3. Launch dynamics, controller, safety, planner, perception, mission, and eval nodes.
4. Accept launch arguments:
  - `scenario`
  - `method`
  - `record`
  - `save_figures`
  - `run_id`
  - `dynamics_backend`
5. Save config snapshots.
6. Save CSV logs.
7. Save `summary.json` and `summary.md`.
8. Start rosbag2 recording when `record:=true`.
9. Generate publication-ready figures.
10. Create `data/results/<timestamp>/` with the exact required structure.

## Prompt 12: Monte Carlo Runner

Implement `monte_carlo_runner.py`.

Requirements:

1. Run multiple scenarios and methods.

2. Support command-line arguments:
  - scenario
  - methods
  - num-runs
  - seed
3. Randomize initial condition, target distribution, sensor noise, and actuation noise.
4. Save each run in its own result directory.
5. Generate combined summary\_table.csv.
6. Generate method\_comparison.png.
7. Report success rate, mean coverage, mean delta-v, mean min distance, and violation count.

## Prompt 13: Basilisk Adapter

Add optional Basilisk backend without breaking existing ROS interfaces.

Requirements:

1. Keep /chaser/odom and /chaser/safe\_control\_command unchanged.
2. Add basilisk\_adapter.py.
3. If Basilisk is not installed, fail gracefully with a clear error message.
4. Add launch argument dynamics\_backend:=hwc or dynamics\_backend:=basilisk.
5. Compare HCW and Basilisk outputs in a validation script.
6. Do not make Basilisk required for the main demo.

## 24. Definition of Done

The project is considered ready for algorithm research when:

1. All packages build with colcon.
2. ROS-only RViz demo works.
3. Gazebo ISS proxy demo works.
4. Fixed waypoint inspection demo works.
5. Greedy NBV inspection demo works.
6. Safety monitor and safety filter work.
7. RViz shows station, chaser, targets, trajectory, FOV, and safety markers.
8. Gazebo shows station and chaser.
9. CSV logs are saved automatically.
10. rosbag2 recording works.
11. Figures are generated automatically.
12. summary.json and summary.md are generated.
13. Monte Carlo runner works.
14. New algorithms can be added without refactoring core interfaces.

## 25. Immediate Command Sequence

Run first:

```
mkdir -p ~/ssei_ws/src
cd ~/ssei_ws
mkdir -p scripts docs data/scenarios data/logs data/bags data/results notebooks
```

Create setup script:

```
nano scripts/setup_ubuntu24_ros2_jazzy.sh
```

Paste the script from Section 15.

Run:

```
chmod +x scripts/setup_ubuntu24_ros2_jazzy.sh
./scripts/setup_ubuntu24_ros2_jazzy.sh
source /opt/ros/jazzy/setup.bash
```

Verify ROS and Gazebo:

```
ros2 doctor
ros2 run demo_nodes_cpp talker
ros2 launch ros_gz_sim gz_sim.launch.py gz_args:="shapes.sdf"
```

Then give Codex this first instruction:

```
Read PROJECT_PLAN.md completely. Implement Phase 1 only. Create the ROS 2 workspace package structure exactly as specified. Do not implement algorithms yet. Ensure colcon build --symlink-install succeeds on Ubuntu 24.04.4 LTS with ROS 2 Jazzy.
```

---

## 26. Final Implementation Strategy

The safest implementation order is:

1. ROS 2 workspace skeleton.
2. ROS-only HCW dynamics.
3. RViz visualization.
4. CSV trajectory/control logger.
5. Gazebo ISS proxy visualization.

6. Inspection targets and coverage.
7. Fixed waypoint inspection.
8. Safety monitor and safety filter.
9. Greedy NBV planner.
10. Paper experiment logging and rosbag.
11. Monte Carlo comparison.
12. Basilisk backend.
13. Advanced safe planner: CBF/MPC/HJI/ADP.

Do not start with Basilisk, MPC, CBF, and Gazebo physics at the same time.

The first research-grade milestone should be:

ROS 2 + RViz + Gazebo + HCW dynamics + fixed waypoint inspection + automatic paper logging.

The first algorithm-grade milestone should be:

Greedy NBV + safety filter + Monte Carlo comparison.

The first publishable milestone should be:

Safety-constrained next-best-view inspection planner with full paper-grade data, visualization, rosbag replay, and method comparison.